PlanER: Cost-Based Optimization of Hybrid Classical–LLM Entity Resolution Plans in a Document Database

D.2.1 Project Full Proposal, COMP 8157 Advanced Database Topics (Fall 2026)

Shashang Patel
School of Computer Science
University of Windsor
Windsor, Canada
patel3cd@uwindsor.ca

Nauman Syed
School of Computer Science
University of Windsor
Windsor, Canada
syed9i@uwindsor.ca

***Abstract*—Entity resolution (ER) is increasingly performed with large language models (LLMs), which match records accurately with little training data but charge for every candidate pair they examine. Classical matchers are nearly free yet weaker on messy data, and today the choice between them is left to the developer. We propose PlanER, which treats ER as a declarative request over a document database and compiles it, through a cost-based optimizer, into a physical plan. The plan selects a blocking access path among MongoDB's B-tree, multikey, full-text, and vector indexes, and a cascade of matchers ranging from string rules and Fellegi–Sunter linkage to local and hosted LLMs. The optimizer estimates each plan's monetary cost, latency, and F1 score from a small labeled sample, and returns the cheapest plan that meets a quality target or the most accurate plan within a budget. We will evaluate PlanER on five public ER benchmarks against LLM-only, classical-only, and embedding-proxy cascade baselines, report the cost–accuracy Pareto frontier of each dataset, and measure how accurately the optimizer predicts the cost and quality of the plans it selects.**

***Index Terms*—Entity resolution, cost-based optimization, large language models, blocking, access path selection, document databases, MongoDB**

I. Introduction and Motivation

Entity resolution (ER) decides which records describe the same real-world entity. A marketplace must recognize that "Apple iPhone 15 Pro 128GB Black" and "iPhone15 Pro (Blk) 128 GB" are one product, and a hospital network must link records of the same patient across clinics. For decades this task was solved with hand-written rules, probabilistic record linkage [1], and supervised classifiers trained on labeled pairs [2], [3].

Two recent developments have changed the situation. First, generative LLMs now match records accurately with little task-specific training [4], [5], and data systems research has begun to expose LLM-powered joins as declarative operators [6], [7]. LLM inference, however, is priced per call, and even after blocking a realistic workload contains tens of thousands to millions of candidate pairs that must be re-resolved whenever a catalog changes. An LLM-only pipeline therefore costs real money with a hosted model or hours of GPU time with a local one. Second, document databases now host vector indexes beside classical ones: since version 8.2, MongoDB Community Edition supports `$search` and `$vectorSearch` without an external engine [8]. A single engine thus offers several access paths for finding candidate pairs, and choosing among access paths is precisely the decision a database system is designed to make.

These developments affect everyone who integrates data under a budget. In current practice a developer picks one matcher and applies it to every pair, which either overpays, when an LLM is used on clean bibliographic data where string similarity already suffices, or loses quality, when cheap rules are used on noisy product titles. Existing systems fall short in complementary ways. Relational and document engines optimize equality joins and leave fuzzy matching to application code. Semantic operator systems such as LOTUS [6] and Abacus [9] optimize LLM-powered operations, but the physical alternatives they consider are model invocations. ER research compares matchers carefully but offers no plan-level optimizer that respects a budget. No system decides which index should generate candidates, which matchers should run, and which pairs deserve an LLM. We argue that this is a cost-based optimization problem in the sense of classical access path selection [10], and we propose to treat it as one.

II. Problem Statement

We consider two collections R and S (or a single collection for deduplication) together with a mapping between their comparable attributes. The user submits a declarative ER request consisting of the collections, the attribute mapping, and an objective, either a quality target θ (minimize cost subject to F1 $\geq \theta$) or a budget B (maximize F1 subject to cost $\leq B$). The system must return a set of entity clusters over $R \cup S$.

A physical plan is a tuple $P = (b, M, \tau, \kappa)$. Here b is a blocking access path that generates candidate pairs, $M = (m_1, \ldots, m_k)$ is a sequence of matchers ordered by

increasing per-pair cost, $\tau = \{(\ell_i, u_i)\}$ are the confidence thresholds of each matcher, and κ is a clustering method. Matcher m_i assigns each pair it receives a score s; the pair is declared a match if $s \geq u_i$, a non-match if $s \leq \ell_i$, and is otherwise escalated to m_{i+1}, with the last matcher deciding every remaining pair. The cost of a plan is

$$C(P) = C_{\text{block}}(b) + \sum_{i=1}^{k} N_i \, c_i + C_{\text{label}}(P), \qquad (1)$$

where N_1 is the number of candidate pairs produced by b, $N_{i+1} = N_i \cdot e_i(\ell_i, u_i)$ with e_i the fraction of pairs that matcher m_i escalates, c_i is the per-pair cost of m_i in dollars or compute seconds, and C_{label} charges for the labeled pairs that supervised matchers and the optimizer itself require.

The problem is difficult for four reasons. Plan quality is unknown without ground truth, so it must be estimated from a small labeled sample whose cost is part of the objective. Recall is bounded by the pair completeness of the access path, so no matcher can recover matches that blocking discarded. The plan space grows combinatorially with access paths, matchers, and thresholds. Finally, independent pairwise decisions can violate transitivity, so cluster quality may differ from pairwise F1. Solving this problem replaces guesswork with a predictable cost–quality trade-off and moves ER from application code into the database, where it can be optimized like any other query.

III. Related Work

The closest line of work reduces the cost of LLM-based entity matching. BatchER [11] groups several pairs into one prompt and selects demonstrations that cover the batch, and ComEM [12] shows that comparing or selecting among several candidates improves on pairwise matching, composing a cheaper LLM for filtering with a stronger one for the final decision. These methods lower the cost of each LLM decision, whereas PlanER lowers the number of pairs that reach an LLM at all. The two are complementary: batching could be applied unchanged to PlanER's LLM tier. CaRL-EM [13] is closest in spirit, since it learns a reinforcement-learning policy that chooses among LLM operators and model sizes to optimize a quality–cost objective. Its action space, however, contains only LLM operators and requires training a policy, while PlanER enumerates a small, interpretable plan space with classical cost-based techniques and includes blocking and non-LLM matchers in that space. Huang et al. [14] benchmark accuracy and cost across 46 LLM configurations; their study quantifies the frontier within the LLM family, and we extend it across paradigms.

Semantic operator systems approach the problem from the database side. LOTUS [6] defines a semantic join whose reference algorithm asks an LLM about every pair, and optimizes it by thresholding cheap embedding-similarity proxies on a sample, with statistical accuracy guarantees relative to that reference. Abacus [9], the optimizer inside Palimpzest [7], searches for the best physical implementation of each

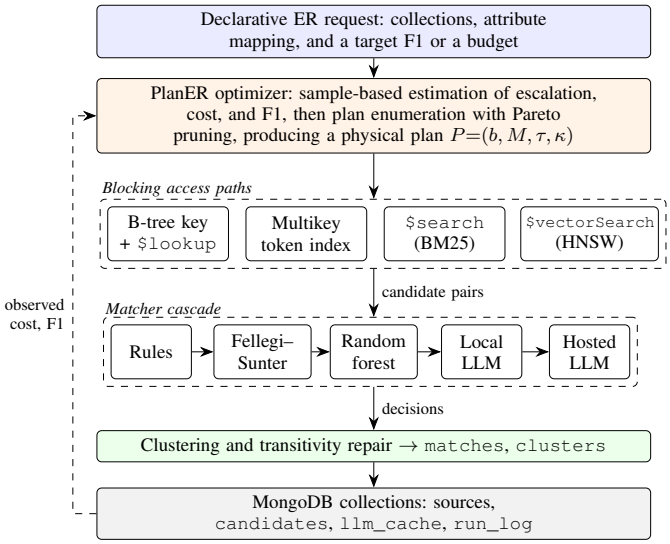


Fig. 1. PlanER architecture. The optimizer compiles a declarative ER request into a physical plan that chooses a blocking access path and a matcher cascade, all executed inside MongoDB.

semantic operator, choosing models and prompting strategies under constrained quality, cost, and latency objectives. Both are the nearest database analogues of our work, and they differ from it in three respects. Their physical operators are model invocations, so a fifty-year-old probabilistic matcher that needs no model at all is outside their plan space. Candidate generation is a nested loop or embedding similarity rather than a choice among the indexes of a storage engine. And LOTUS's guarantees are relative to an LLM reference, so when the LLM itself errs, the guarantee does not measure true accuracy, whereas we evaluate against ground truth. We adopt the LOTUS embedding-proxy cascade as a baseline to quantify what classical operators add.

Peeters et al. [5] compare hosted and open LLMs across prompt designs and find that the best prompt depends on the model and dataset, and Narayan et al. [4] first showed that foundation models perform matching in few-shot settings. Both evaluate accuracy on pre-generated candidate pairs and leave cost and blocking aside, which is exactly the gap PlanER targets. On the learned side, Ditto [3] fine-tunes pre-trained transformers and remains among the most accurate supervised matchers, and DeepBlocker [15] explores deep learning for blocking. Both assume that labeled data and GPU training are available; PlanER instead treats such operators as plan alternatives whose cost includes the labels they consume.

Model cascades predate entity resolution. FrugalGPT [16] showed that querying cheaper LLMs first and escalating only when a learned scorer judges an answer unreliable can approach the quality of the best model at a fraction of its cost. PlanER inherits the escalation idea but differs in two ways that matter for data management. Its tiers are heterogeneous rather than a ladder of LLM services, so the cheapest tiers cost several orders of magnitude less per pair than any model call, which changes where the optimal thresholds lie. In addition,

its cascade runs over a set of candidate pairs produced by a database access path rather than over independent queries, so the optimizer must reason jointly about how many pairs blocking produces and how many of them survive each tier, as captured by Eq. (1).

Blocking has its own extensive literature. Papadakis et al. [17] survey blocking and filtering techniques and show that each one trades pair completeness against the number of comparisons it leaves. These techniques are usually implemented as standalone algorithms over in-memory data. PlanER instead realizes blocking with indexes that a production document database already maintains, which exposes the trade-off to an optimizer and allows us to measure the latency and index footprint of each method inside the engine rather than in isolation. Magellan [2] guides a human developer through this workflow, but the developer still chooses every step; PlanER lets an optimizer make these choices from a stated objective. Across all of these works, no system places classical, supervised, and LLM matchers and several index-based blocking strategies in one optimizer driven by a user-stated budget, which is the gap our project addresses.

IV. Novelty and Contributions

The central assumption we break is that the physical alternatives for an LLM-powered matching operator are themselves models. Every cost-aware system discussed above chooses among LLMs of different sizes, prompting strategies, or embedding proxies. We argue that the cheapest correct operator is often not a model at all: on many pairs, a string rule or a probabilistic linkage model decides with near certainty for essentially zero cost, and only the residual ambiguous pairs need an LLM. We break two further assumptions. The first is that candidate generation is a fixed input to matching; in PlanER it is an access path chosen by the optimizer from the indexes the database already maintains, and its pair completeness bounds every downstream decision. The second is that pairwise accuracy is the right measure; we evaluate the clusters that users actually consume and test whether inexpensive clustering repairs transitivity violations without additional LLM calls.

Our project makes four contributions. First, it formulates budget-constrained ER as cost-based plan selection over heterogeneous physical operators inside a document database, with labeling cost included in the objective. Second, it contributes the PlanER optimizer, which estimates the escalation rate, cost, and quality of each plan from a small stratified sample and selects a plan by enumeration with Pareto pruning. Third, it delivers an implementation in which blocking, cascading, caching, and clustering run as MongoDB aggregation pipelines over four kinds of index. Fourth, it provides an evaluation reporting the cost–accuracy frontier of each dataset, the error between predicted and observed cost and F1, and a categorized failure analysis. The expected insight is a measured answer to when an LLM is worth paying for in ER, and evidence that a database optimizer, rather than the developer, can make that decision. The project exercises declarative abstraction, logical–physical data independence, since one request can execute under many equivalent plans, cost-based optimization, and indexing as access path selection.

V. System Design

Fig. 1 shows the architecture. Each record is a MongoDB document that keeps its original attributes with derived fields: normalized strings, a token array, a blocking key, and a 384-dimensional sentence embedding. Candidate pairs, matcher scores, cached LLM verdicts keyed by a pair hash, matches, clusters, and an execution log have their own collections, and the ER request itself is stored as a document.

Every request has the same logical plan of blocking, matching, and clustering; the physical plan decides how each step runs. The four blocking access paths are equality on a blocking key through a B-tree index and `$lookup`, shared rare tokens through a multikey index, the top-k lexical neighbors from `$search`, and the top-k semantic neighbors from `$vectorSearch`. Matchers, in order of per-pair cost, are similarity rules, an unsupervised Fellegi–Sunter model fitted by expectation–maximization, a random forest over similarity features, a local LLM, and a hosted LLM, each producing a calibrated score.

Optimization has three steps. PlanER first draws a few hundred candidate pairs stratified by similarity score, so that rare matches are represented, and charges their labeling to C_{label}. It then runs the matchers on this sample, obtaining for each one its per-pair cost and, for any thresholds, its escalation rate e_i and the precision and recall of decided pairs; access paths are profiled for candidate count, latency, and pair completeness. Finally, because matchers have a fixed cost order, a plan is an access path, a subset of matchers, and a threshold grid; plans dominated in both estimated cost and F1 are pruned, and the Pareto set is searched for the plan meeting the objective under Eq. (1). The plan then executes as aggregation pipelines, with LLM verdicts read from and written to the cache and `explain()` output recorded for blocking queries. Matches form a graph whose connected components are the initial clusters; a repair step splits components containing confident non-matches. Observed cost and F1 are logged, which lets us measure the optimizer's estimation error directly.

VI. Implementation Plan

PlanER will be written in Python 3.11 with PyMongo against MongoDB 8.2 Community Edition and its `mongot` search process, deployed in Docker so that both members run an identical environment. Embeddings come from the all-MiniLM-L6-v2 sentence transformer, string similarities from RapidFuzz, and the random forest from scikit-learn; Fellegi–Sunter is implemented directly over pairs fetched from MongoDB. Llama 3.1 8B and Qwen 2.5 7B run locally through Ollama with greedy decoding and a fixed prompt, one low-cost hosted model provides real per-token prices, and Ditto [3] is an optional tier trained on a cloud GPU. A Streamlit interface lets a user submit a request, inspect the chosen plan, and move the target F1 or budget while cost and quality

update; it is also our demonstration. We use benchmarks with full ground truth [18]: DBLP–ACM (about 2.6K and 2.3K records) is clean bibliographic data, Abt–Buy (1.1K per side) and Amazon–Google (1.4K and 3.2K) are noisy product data, and DBLP–Scholar (2.6K and 64K records, about 168 million possible pairs) stresses blocking at scale. WDC Products [19] adds unseen entities. Baselines are an LLM-only plan, the best single classical matcher, and a LOTUS-style embedding-proxy cascade [6]. We report pairwise and cluster F1, dollars, compute seconds, labeling cost, LLM calls, pair completeness and reduction ratio, transitivity violations, and the optimizer's prediction error.

VII. Risks and Mitigation

Search and vector search in MongoDB Community Edition are in public preview, so `mongot` may be unstable. Access paths sit behind a common interface, and the vector path can move to the free MongoDB Atlas tier, which exposes the same `$vectorSearch` stage. Local LLMs may be slow on laptops; we cache every verdict, send only escalated pairs to LLMs, use DBLP–Scholar mainly for blocking experiments, and fall back to smaller models if needed. The optimizer's estimates may be poor when samples are small or matches rare; stratified sampling mitigates this, we will vary the sample size to show how error shrinks, and poor calibration would itself be a reportable finding. LLM outputs can vary and models may have seen public benchmarks; greedy decoding and caching make runs repeatable, and WDC Products limits contamination. Training, estimation, and test splits will be disjoint to prevent leakage. To control scope, the minimum viable project is three datasets, three access paths, three classical matchers with one local LLM, and the optimizer; WDC Products, Ditto, the hosted model, and cluster repair are extensions.

VIII. Timeline and Milestones

In phase one (October 5–18) we deploy MongoDB with `mongot`, load and normalize the datasets, build the four indexes, and deliver pair completeness and latency for every access path. In phase two (October 19–November 1) we implement the classical matchers, integrate the local LLMs with the verdict cache, and deliver accuracy and per-pair cost for every matcher. In phase three (November 2–15) we implement sampling, estimation, and plan enumeration, and deliver the working optimizer with a first predicted-versus-observed comparison. In phase four (November 16–29) we run the full study against all baselines, add cluster repair, and deliver the Pareto frontiers and failure analysis. In phase five (November 30 to the end of term) we complete the demonstration, final report, and presentation. One member leads blocking and storage and the other leads matching and optimization, and both contribute to every phase.

References

[1] I. P. Fellegi and A. B. Sunter, "A theory for record linkage," *J. Amer. Statist. Assoc.*, vol. 64, no. 328, pp. 1183–1210, 1969.

[2] P. Konda et al., "Magellan: Toward building entity matching management systems," *Proc. VLDB Endow.*, vol. 9, no. 12, pp. 1197–1208, 2016.

[3] Y. Li, J. Li, Y. Suhara, A. Doan, and W.-C. Tan, "Deep entity matching with pre-trained language models," *Proc. VLDB Endow.*, vol. 14, no. 1, pp. 50–60, 2020.

[4] A. Narayan, I. Chami, L. Orr, S. Arora, and C. Ré, "Can foundation models wrangle your data?" *Proc. VLDB Endow.*, vol. 16, no. 4, pp. 738–746, 2022.

[5] R. Peeters, A. Steiner, and C. Bizer, "Entity matching using large language models," in *Proc. 28th Int. Conf. Extending Database Technology (EDBT)*, 2025, pp. 529–541.

[6] L. Patel, S. Jha, M. Pan, H. Gupta, P. Asawa, C. Guestrin, and M. Zaharia, "Semantic operators and their optimization: Enabling LLM-based data processing with accuracy guarantees in LOTUS," *Proc. VLDB Endow.*, vol. 18, no. 11, pp. 4171–4184, 2025.

[7] C. Liu et al., "Palimpzest: Optimizing AI-powered analytics with declarative query processing," in *Proc. Conf. Innovative Data Systems Research (CIDR)*, 2025.

[8] MongoDB, Inc., "Public preview: MongoDB Community Edition now offers native full-text and vector search," Sep. 2025. [Online]. Available: https://www.mongodb.com/products/updates/

[9] M. Russo, C. Liu, S. Sudhir, G. Vitagliano, M. Cafarella, T. Kraska, and S. Madden, "Abacus: A cost-based optimizer for semantic operator systems," *Proc. VLDB Endow.*, vol. 19, no. 5, pp. 1060–1073, 2026.

[10] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price, "Access path selection in a relational database management system," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 1979, pp. 23–34.

[11] M. Fan, X. Han, J. Fan, C. Chai, N. Tang, G. Li, and X. Du, "Cost-effective in-context learning for entity resolution: A design space exploration," in *Proc. 40th IEEE Int. Conf. Data Engineering (ICDE)*, 2024, pp. 3696–3709.

[12] T. Wang et al., "Match, compare, or select? An investigation of large language models for entity matching," in *Proc. 31st Int. Conf. Computational Linguistics (COLING)*, 2025, pp. 96–109.

[13] C. Guo, M. C. A. Klein, and Z. Huang, "CaRL-EM: Cost-aware reinforcement learning for entity matching with LLMs," in *Proc. 64th Annu. Meeting Assoc. Computational Linguistics (ACL)*, 2026, pp. 27281–27293.

[14] D. Huang, P. Pai, and Z. Wang, "Entity matching with LLMs at scale: Accuracy, cost, and reasoning effects," in *Proc. Pacific-Asia Conf. Knowledge Discovery and Data Mining (PAKDD)*, LNCS vol. 16603, Springer, 2026.

[15] S. Thirumuruganathan et al., "Deep learning for blocking in entity matching: A design space exploration," *Proc. VLDB Endow.*, vol. 14, no. 11, pp. 2459–2472, 2021.

[16] L. Chen, M. Zaharia, and J. Zou, "FrugalGPT: How to use large language models while reducing cost and improving performance," arXiv:2305.05176, 2023.

[17] G. Papadakis, D. Skoutas, E. Thanos, and T. Palpanas, "Blocking and filtering techniques for entity resolution: A survey," *ACM Comput. Surv.*, vol. 53, no. 2, pp. 31:1–31:42, 2020.

[18] H. Köpcke, A. Thor, and E. Rahm, "Evaluation of entity resolution approaches on real-world match problems," *Proc. VLDB Endow.*, vol. 3, no. 1, pp. 484–493, 2010.

[19] R. Peeters, R. C. Der, and C. Bizer, "WDC Products: A multi-dimensional entity matching benchmark," in *Proc. 27th Int. Conf. Extending Database Technology (EDBT)*, 2024, pp. 22–33.